An Efficient Malware Detection Framework for Enhancing Software Security in Resource-Constrained Systems

Govind P. Gupta, *Member, IEEE*, Prabhat Kumar, *Member, IEEE*, and Ahamed Aljuhani

***Abstract*—The increasing adoption of Industrial Internet of Things (IIoT) networks has introduced new security challenges, particularly to ensure software security against evolving malware threats. IIoT systems rely on interconnected edge, cloud, and embedded devices, which are highly vulnerable to malware attacks that exploit software vulnerabilities, propagate across networks, and compromise industrial operations. However, existing malware detection approaches often struggle with resource constraints, high-dimensional feature spaces, and the need for real-time adaptability, making them inefficient for large-scale IIoT deployments. To address these challenges, this article presents "gray wolf optimization and particle swarm optimization (GWPSO)-graph neural network-based detection model (GAMD)," a resource-efficient malware detection framework designed to enhance software security in IIoT networks. The framework integrates a hybrid metaheuristic feature selection algorithm—GWPSO—to identify the most discriminative and computationally efficient features, reducing processing overhead while maintaining high detection accuracy. These features are then analyzed by the GAMD, which leverages graph convolutional networks (GCNs) and attention mechanisms to model malware propagation behaviors and uncover complex attack patterns in IIoT environments. Empirical evaluations on two open-source malware datasets, CIC-MalDroid-2020 and CIC-MalMem-2022, demonstrate that the proposed model achieves detection accuracy above 98.41% while significantly reducing CPU usage by 47%, memory footprint by 57%, and training time by 67%. The results highlight GWPSO-GAMD's ability to provide scalable, real-time malware detection for resource-constrained IIoT systems, advancing the vision of secure, intelligent, and resource-aware IIoT networks.**

***Index Terms*—Android, graph neural network (GNN), malware detection system, mobile ecosystem.**

I. Introduction

IN RECENT years, the Industrial Internet of Things (IIoT) has become an essential enabler of smart manufacturing, automated industrial control, and real-time monitoring systems. These interconnected IIoT devices often rely on Android-based applications and edge computing frameworks for seamless remote access, diagnostics, and data collection [1]. However, this growing interconnectivity also expands the attack surface, as malware designed for mobile environments can now exploit software vulnerabilities within IIoT networks, compromising critical infrastructure and industrial control systems (ICS) [2]. The challenge is further exacerbated by the resource-constrained nature of IIoT devices, which operate under strict computational, memory, and energy limitations, making traditional malware detection mechanisms impractical for real-world deployment. A key security concern arises from the increasing convergence of Android-driven software with IIoT applications, where malware infiltration can occur through firmware updates, third-party applications, and unauthorized remote access [3]. Attackers leverage these entry points to execute malicious operations, such as exfiltrating industrial data, disrupting IIoT system processes, or hijacking edge computing resources [4]. In distributed IIoT environments, where devices continuously exchange information, the detection of such malware is particularly challenging due to dynamic workloads, evolving threat landscapes, and the computational overhead of real-time security analysis [5].

To ensure software security in IIoT, malware detection has become a critical research area, requiring solutions that are accurate and resource-efficient. Existing detection techniques fall into two major categories: 1) static analysis and 2) dynamic analysis [6], [7]. Static detection relies on signature-based methods that match known malware patterns but struggle against zero-day malware and obfuscation techniques [8]. Dynamic detection, on the other hand, monitors behavioral patterns using machine learning (ML), deep learning (DL), and graph neural networks (GNNs) to uncover hidden malicious behaviors in IIoT environments. However, while dynamic approaches provide better detection accuracy, they require significant computational resources, making them infeasible for energy-constrained IIoT deployments [9], [10].

To mitigate the computational overhead associated with dynamic malware detection, feature selection techniques play a crucial role in resource optimization. High-dimensional malware datasets often contain redundant, irrelevant, or noisy features, which can increase processing time and energy consumption while degrading model performance [11]. Using intelligent feature selection mechanisms, such as hybrid metaheuristic approaches, it is possible to identify the most

Received 11 February 2025; revised 6 June 2025 and 8 July 2025; accepted 4 August 2025. Date of publication 6 August 2025; date of current version 24 October 2025. This work was supported by the SERB (CRG) Project which is funded by the Anusandhan National Research Foundation (ANRF), India, under Grant CRG/2023/003064. *(Corresponding author: Govind P. Gupta.)*

Govind P. Gupta is with the Department of Information Technology, National Institute of Technology, Raipur 492010, India (e-mail: gpgupta.it@nitrr.ac.in).

Prabhat Kumar is with the Department of Software Engineering, LUT School of Engineering Science, LUT University, 53850 Lappeenranta, Finland (e-mail: prabhat.kumar@lut.fi).

Ahamed Aljuhani is with the Department of Information Technology, University of Tabuk, Tabuk 71491, Saudi Arabia (e-mail: a_aljuhani@ut.edu.sa).

Digital Object Identifier 10.1109/JIOT.2025.3596313

discriminative yet lightweight feature subset for malware classification. This not only reduces memory and CPU usage, but also enhances the scalability of malware detection models, making them better suited for IIoT environments with limited computing power [12]. Thus, integrating optimized feature selection with dynamic malware detection methods ensures a balance between detection accuracy and computational efficiency, enabling real-time, resource-aware security solutions for IIoT ecosystems. Feature selection techniques are mainly categorized into two types: wrapper and filter method [13]. The wrapper technique tests all possible combinations of features in the algorithm used to train the model and select the set of features that provides the best solution. The wrapper method consists of three elements: classifiers, evaluation criteria, and search algorithm. This method becomes computationally costly when dealing with a large number of features [14]. On the other hand, the filter technique takes less computational time. In the filter method, the importance of the feature is calculated using the evaluation function. The features are then ranked according to feature weight. The highest weighted features are selected. This method does not consider the accuracy computed by the classifier for a particular set of features, so the method becomes less effective [15].

Metaheuristic algorithms-based feature selection techniques are wrapper methods and use an optimization approach to find suitable features. They give better performance as they do not use the entire search space. For example, if there are n features then the total possible numbers of solutions are $2n$. Metaheuristic methods are classified into two types as: 1) single-objective and 2) multiobjective methods. Feature selection problem is a multiobjective problem [16]. Among the prevalent metaheuristic algorithms for optimal feature selection are gray wolf optimization (GWO), the genetic algorithm, particle swarm optimization (PSO), and the gravitational search algorithm, emphasizing the crucial balance between exploration—discovering new possibilities—and exploitation—refining existing solutions. This balance is pivotal for enhancing algorithmic performance, often achieved by merging multiple algorithms [17]. The proposed hybrid model in this study integrates PSO and GWO in a unique co-evolutionary approach, combining PSO's exploration strengths with GWO's exploitation capabilities to efficiently identify an optimal feature subset, aiming for dataset minimization while maximizing accuracy.

The PSO approach is inspired by the behavior of birds and fish in the search for food. It consists of three main global variables that need to be monitored: 1) global value; 2) target value; and 3) stopping value. Every particle in PSO consists of a position vector [18]. GWO is based on the hunting behavior of gray wolf. There are four types of gray wolf: 1) alpha; 2) beta; 3) delta; and 4) omega wolves. It is assumed that hunting is performed by alpha, beta, and delta. The best solution is represented by alpha wolves [19]. In this work, we extend our malware detection methodology to include a graph-based detection framework alongside dynamic feature selection. The PSO and GWO algorithms inspire our approach, leveraging collective behavior and hunting strategies, respectively, for effective feature optimization. By integrating a GNN model, we enhance the system's ability to analyze complex interrelations within the data, aiming for superior detection capabilities. This novel integration aims to refine feature selection and boost the efficacy of ensemble learning techniques, addressing the intricacies of malware threats in evolving digital environments (Table I). The main contributions of this article are as follows.

TABLE I
ABBREVIATION USED IN THIS ARTICLE

Feature Selection Methods	Classification Algorithm
Grey Wolf Optimizer (GWO)	Graph Convolutional Network (GCN)
Particle Swarm Optimization (PSO)	Graph Attention Network (GAT)
Hybrid of Grey wolf optimizer and Particle Swarm Optimization (GWPSO)	Graph Android Malware Detector (GAMD)

1) A graph-based malware detection framework named "GWO and PSO (GWPSO)-GNN-based detection model (GAMD)" is proposed for securing mobile ecosystems.
2) A hybrid feature selection technique named "GWPSO" is proposed by combining GWO and PSO. In addition, a modified fitness function is designed to select the most relevant features, optimizing both the reduction of feature space and the improvement of detection accuracy.
3) The optimized features are used to design a GAMD that integrates graph convolutional network (GCN) layers with graph attention network (GAT) layers. This setup is adept at processing the optimized feature set and capturing complex patterns characteristic of malware, showcasing the potential of graph-based models in cybersecurity realms.
4) The performance of the proposed "GWPSO-GAMD" framework is evaluated and compared with baselines using two publicly available datasets, namely, CIC-MalDroid-2020 and CIC-MalMem-2022 datasets.

The remainder of the article is organized as follows. Section II presents the work that is relevant to this study. Section III presents proposed methods for identifying malware on Android. Section IV presents the results of the experiments. Finally, Section V presents the conclusion with a future research perspective.

II. RELATED STUDY

Recently, several researchers have proposed various Android malware detection frameworks based on different feature selection techniques. For example, Dhalariav and Gandotra [20] designed an Android malware detection framework using the chi-square feature selection technique. Their approach used the AndroMD dataset that has 675 features, 3547 instances, and two classes of benign and malware to predict. Furthermore, the dataset was split into 1:9 ratios. The 90% dataset was used for training and 10% for testing. Their model used five classifiers as the base and further combined them using the stacking ensemble learning method. Among all the ML techniques, KNN_RF achieved the highest accuracy of 98.02%. Fatima et al. [21] proposed Android malware detection in two units. First, the feature was extracted using the Androguard tool and the features were selected

TABLE II
COMPARISON OF RELATED ANDROID MALWARE DETECTION APPROACHES

Author(s)	Detection Technique	Dataset and Features	Best Accuracy (%)	Limitations
Dhalaria *et al.* [20]	Chi-square + KNN_RF	AndroMD (675 features, 3547 instances)	98.02	Limited scalability and traditional ML pipeline
Fatima *et al.* [21]	Genetic Algorithm + SVM/NN	Androguard features (33/40 selected)	Not specified	Accuracy reduced after feature selection; limited robustness
Mahindru *et al.* [22]	Hybrid Framework	Large-scale dataset (141 malware categories, 262K apps)	Not specified	No detailed deep learning integration
Al Sarah *et al.* [23]	RFE + Ensemble ML (LightGBM, etc.)	Drebin (imbalanced, 55 features selected)	99.5	Limited to static features; ML-based
Zhu *et al.* [24]	Static + Data Flow Features with Deep Stacking Ensemble	Real-time dataset with static and dynamic features	89.07 (static), 94.92 (data flow)	Higher complexity; not optimized for edge/resource-constrained use
Nguyen *et al.* [25]	D2MD + Hybrid Deep Learning + CTGAN	Cloud-based (227 dynamic + 12 static features)	99.42	Cloud-centric, may not generalize well to edge/IIoT systems
Sharma *et al.* [26]	Opcode-based Static Detection (MOSDroid)	51K+ APKs (25,990 obfuscated samples)	98.41	Focused on static features; lacks dynamic behavior modeling

using a genetic algorithm. Selected features were given to ML algorithms, such as support vector machine (SVM) and neural networks based on the performance that was evaluated. The classification accuracy was reduced after employing the feature selection technique. Their approach used SVM and the neural network as base learners and selected 33 and 40 features, respectively. Furthermore, Mahindru and Sangal [22] designed an effective Android malware detection framework. In this article, distinct malware-infected Android apps were collected that had 141 distinct categories and 2 62 197 apps.

Al Sarah et al. [23] used static analysis to detect Android malware. The dataset used is the imbalanced Drebin dataset, which goes through data preprocessing. First, the dataset is balanced by randomly shuffling the dataset. Then relevant features are selected by using two methods which are recursive feature elimination (RFE) with cross-validation and RFE. The optimal number of selected features is 55. This article uses eight ML algorithms which are LightGBM, XGBoost, random forest (RF), logistic regression (LR), SVM Linear Vector, Decision Tree, Gaussian naive Bayes (NB), and gradient boosting. The experiment shows that an ensemble method performs better than traditional ML methods. The results show that LightGBM has achieved the highest accuracy of 99.5%. Zhu et al. [24] proposed a stacking ensemble framework for Android malware detection that combines static analysis and DL. This approach has three units. First, feature extractor is used to extract static features, such as permission, sensitive APIs, monitoring system events, and permission rate. This static feature can be replaced by data flow. These extracted features are used by stacking an ensemble deep learner component which makes a base classifier and fusion of classifiers. The real-time dataset is used in this article. With the static feature, the average detection accuracy is 89.07% and the data flow feature is 94.92%. Nguyen et al. [25] proposed a cloud-based malware dataset comprising 227 dynamic and 12 static features, specifically designed to support binary and multiclass classification tasks. The D2MD framework was introduced, integrating hybrid DL techniques to enhance detection performance in cloud environments. To address data imbalance, the study employed conditional tabular GANs (CTGAN), leading to improved detection rates for rare malware categories. The approach achieved 99.42% accuracy in binary classification and demonstrated effective scalability for cloud-integrated IDS systems. Sharma et al. [26] introduced MOSDroid, a static Android malware detection method designed to resist code obfuscation techniques by leveraging multisets of encoded opcode sequences (MOS). The method encodes method-level opcode sequences and employs a two-stage feature selection process to enhance model efficiency and robustness. Experimental evaluation on over 51 000 APKs, including 25 990 obfuscated samples, demonstrated an accuracy of 98.41% and an area under the curve (AUC) of 99.45%, confirming the model's resilience to diverse obfuscation strategies. Table II shows the limitation of existing approaches in Android malware detection for resource constrained environment.

III. PROPOSED GWPSO-GAMD FOR SECURING MOBILE ECOSYSTEMS IN THE INDUSTRIAL METAVERSE

This section outlines the proposed graph-based malware detection framework and is shown in Fig 1. The proposed GWPSO-GAMD includes three key stages: 1) preprocessing the initial dataset to address missing and categorical values; 2) feature selection using the proposed GWPSO technique; and 3) designing a novel GAMD for enhanced malware detection. Each phase of the framework is detailed below, explaining its role in the overall malware detection process.

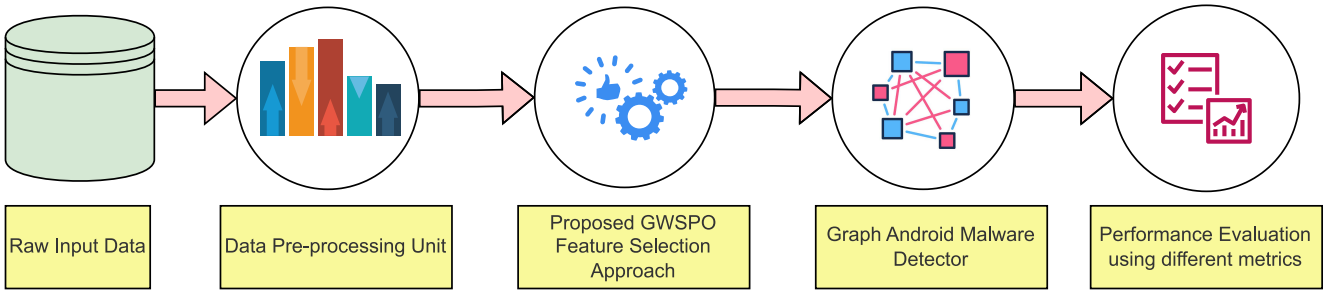

Fig. 1. Flowchart for the proposed approach.

A. Data Preprocessing Unit

This subsection includes the details of the preprocessing steps used in the GWPSO-GAMD framework.

1) Checking for Missing Values: Addressing missing values in datasets, such as CIC-MalDroid-2020 and CIC-MalMem-2022 is crucial for maintaining data integrity and ensuring accurate malware detection models. Missing data can distort the analysis, which can lead to incorrect classifications. The initial step involves identifying missing values across features, followed by employing techniques, such as imputation to fill these gaps based on the data's distribution or known relationships. For example, missing entries in the "API call frequency" feature have been imputed using median values, ensuring the model has a complete view of application behaviors which is critical for distinguishing between benign and malicious instances.

2) Encoding Categorical Values: Encoding categorical values is equally important for preparing the dataset for ML algorithms, which typically require numerical input. This process transforms non-numerical or categorical data into a machine-readable or integer format without losing the intrinsic information. In this article, we have used one-hot encoding, where each category is transformed into a new binary column. For instance, the "Network Type" feature, categorizing network connections as WiFi or Mobile, has been split into two columns: one representing WiFi and the other "Mobile," with 0 or 1 indicating the presence of each category. This transformation allows the model to accurately factor in the network type's impact on malware behavior.

B. Proposed GWPSO for Feature Selection

The feature selection process is designed to eliminate redundant and irrelevant features, taking the entire set of dataset features as input and obtaining an optimal subset for use. This step is pivotal in boosting model accuracy and efficiency by minimizing the dataset's complexity. Our proposed GWPSO-GAMD framework first incorporates a hybrid metaheuristic wrapper-based feature selection technique that merges the strengths of PSO and GWO. This approach specifically targets the acquisition of a refined subsets of features, facilitating the classification of Android malware. The methodology behind this feature selection process is further detailed in the following discussion.

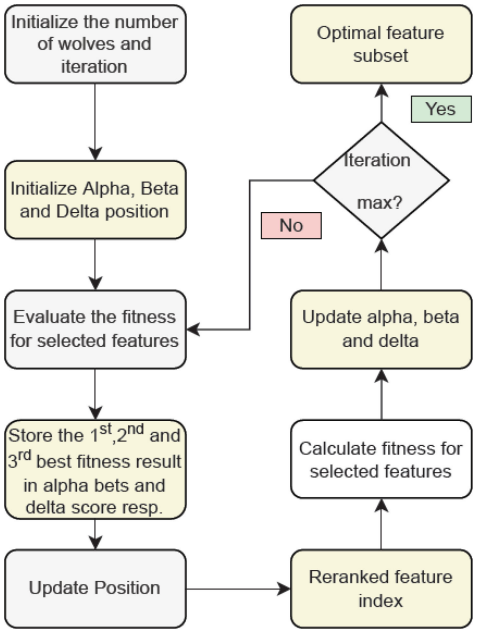

Fig. 2. Flowchart for Grey Wolf Optimizer feature selection approach.

1) Grey Wolf Optimizer-Based Feature Selection: This approach is based on how gray wolves hunt. GWO consists of four levels as follows.

1) Alpha (α): male and female wolves are the leaders of the pack and takes decision of waking up, sleeping, and hunting.
2) Beta (β): beat male or female is the best candidate to replace. They give feedback and help Alpha in decision-making.
3) Delta (δ): they obey alpha and beta wolves and go hunting.
4) Omega (ω): they are weakest wolves and obey other wolves.

Fig. 2 shows the flowchart for the feature selection method using GWO algorithms. Alpha is the first best solution in the GWO algorithm. Beta and delta are the second and third-best solutions, respectively. Omega follows alpha, beta, and delta. Throughout the hunting process gloves encircle the prey which is calculated as

$$\vec{X}(t+1) = \vec{X}_p(t) + \vec{\mathcal{A}} \odot \vec{\mathcal{D}} \tag{1}$$

where $\overrightarrow{X}_p$ is the position vector of prey, t is the iteration number, $\overrightarrow{\mathcal{A}}$ is the coefficient vector. We can compute $\overrightarrow{\mathcal{D}}$ as

$$\overrightarrow{\mathcal{D}} = \left| \overrightarrow{C} \odot \overrightarrow{X}_p(t) - \overrightarrow{X}(t) \right| \tag{2}$$

where $\overrightarrow{C}$ is the coefficient vector. $\overrightarrow{\mathcal{A}}$ and $\overrightarrow{C}$ are computed as

$$\overrightarrow{\mathcal{A}} = 2\overrightarrow{a} \odot \overrightarrow{r}_1 - \overrightarrow{a} \tag{3}$$

where $\overrightarrow{a}$ is the set vector which decreases linearly from 2 to 0 as the iteration increases which is updated using the following:

$$\overrightarrow{a} = 2 - t \odot \left(\frac{2}{\text{max\_iter}} \right) \tag{4}$$

$$\overrightarrow{C} = 2 \odot \overrightarrow{r}_2 \tag{5}$$

where $\overrightarrow{r}_1$ and $\overrightarrow{r}_2$ are random vectors and their value range from 0 to 1. The three best solutions are found in each iteration and there position is changed by using the following:

$$\overrightarrow{X}_{t+1} = \frac{\overrightarrow{X}_1 + \overrightarrow{X}_2 + \overrightarrow{X}_3}{3} \tag{6}$$

$\overrightarrow{X}_1$, $\overrightarrow{X}_2$, and $\overrightarrow{X}_3$ are the position vector of alpha, beta and delta resp. which are calculated as

$$\overrightarrow{X}_1 = \overrightarrow{X}_\alpha - \overrightarrow{\mathcal{A}}_1 \left(\overrightarrow{\mathcal{D}}_\alpha \right) \tag{7}$$

$$\overrightarrow{X}_2 = \overrightarrow{X}_\beta - \overrightarrow{\mathcal{A}}_1 \left(\overrightarrow{\mathcal{D}}_\beta \right) \tag{8}$$

$$\overrightarrow{X}_3 = \overrightarrow{X}_\gamma - \overrightarrow{\mathcal{A}}_1 \left(\overrightarrow{\mathcal{D}}_\gamma \right) \tag{9}$$

where $\overrightarrow{\mathcal{D}}_\alpha$, $\overrightarrow{\mathcal{D}}_\beta$, $\overrightarrow{\mathcal{D}}_\gamma$ are the distance vectors for each wolf which are calculated as

$$\overrightarrow{\mathcal{D}}_\alpha = \left| \overrightarrow{C}_1 \odot \overrightarrow{X}_\alpha - \overrightarrow{X} \right| \tag{10}$$

$$\overrightarrow{\mathcal{D}}_\beta = \left| \overrightarrow{C}_2 \odot \overrightarrow{X}_\beta - \overrightarrow{X} \right| \tag{11}$$

$$\overrightarrow{\mathcal{D}}_\gamma = \left| \overrightarrow{C}_3 \odot \overrightarrow{X}_\gamma - \overrightarrow{X} \right| \tag{12}$$

2) Particle Swarm Optimization Based Feature Selection: Due to its efficient nonlinear approach, it is a popular optimization algorithm. It is a population-based optimization that is inspired by the behavior of flying birds. The particles in PSO have position and velocity vectors. Each particle follows other particles around it which are local best $X_l^i(t)$ and the particle leading all others is global best $X_g^i(t)$. Velocity and position are updated as follows:

$$v^i(t+1) = v^i(t) * w + c_1 * r_1 * \left(X_g^i(t) - X^i(t) \right) + c_2 * r_2 * \left(X_l^i(t) - X^i(t) \right) \tag{13}$$

$$X^i(t+1) = X^i(t) + v^i(t+1) \tag{14}$$

$X_g^i(t)$ is the value of global best which has maximum accuracy. $X_l^i(t)$ is the local best for particle. c_1 and c_2 are constant which has fixed value 1. c_1 is cognition coefficient and c_2 is the social coefficient component. w is inertial weight. r_1 and r_1 is random number valued between 0 and 1. Fig. 3 shows the flowchart for the feature selection method using the PSO feature selection algorithm.

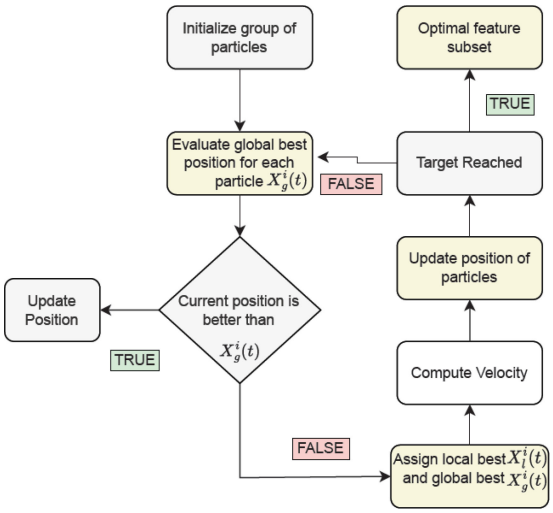

Fig. 3. Flowchart for PSO feature selection approach.

Algorithm 1 Training Procedure for the Proposed GWPSO Feature Selection Approach

Input: Initialize population n wolves, number of iteration: max_*iter*, number of features: d, calculated coefficient vector, random vector and set vector.
Output: Optimal subset of feature
Preprocess the dataset
Evaluate fitness function and get α, β, γ solutions **for**$t = 1 : max\_iter$ **do**
Calculate $\overrightarrow{\mathcal{A}}$, $\overrightarrow{a}$, $\overrightarrow{C}$ using Eqs. (3), (4) and (5) **for**$i = 1 : n$ **do**
$i = 1 : d$
Calculate $\overrightarrow{\mathcal{D}}_\alpha$, $\overrightarrow{\mathcal{D}}_\beta$, $\overrightarrow{\mathcal{D}}_\gamma$ using Eqs. (15), (16) and (17)
Calculate $\overrightarrow{X}_\alpha$, $\overrightarrow{X}_\beta$, $\overrightarrow{X}_\gamma$ using Eqs. (7), (8) and (9)
Update position using (19)
Calculate fitness and get updated α, β, γ solutions

3) Proposed GWPSO Feature Selection Technique for Android Malware Detection: This subsection presents GWPSO metaheuristic-based hybrid feature selection for Android malware detection. This algorithm uses two filter metaheuristic feature selection methods. The proposed feature selection algorithm is shown in Algorithm 1. The subset of features obtained impacts the accuracy of the model.

Hybrid GWPSO increases the algorithm performance by increasing the algorithm's ability to explore GWO and exploit PSO. Due to the exploration ability of GWO particles are directed to the partially improved position and then to a random position. But as a disadvantage, running time of the algorithm is increased. The first step of the algorithm is to initialize a random population of n wolves of length d which is the number of features in the dataset. It contains binary value 0's and 1's. 1 represents selected relevant features, and others are ignored and represented by 1. Then the distance vector for

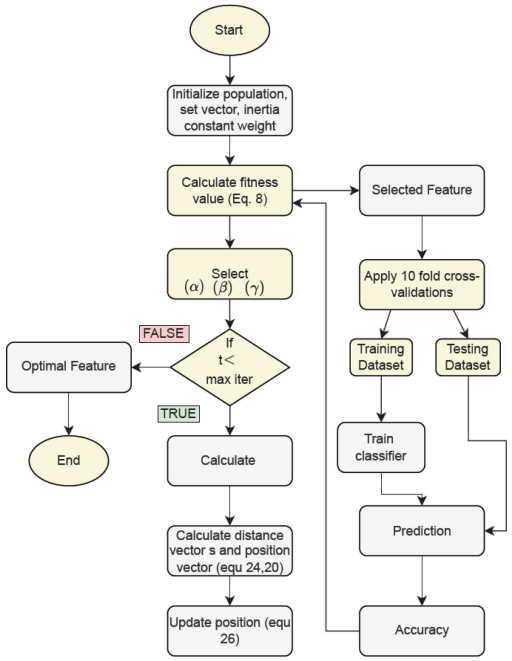

Fig. 4. Flowchart for the proposed GWPSO feature selection approach.

alpha, beta, and delta is calculated as

$$\overrightarrow{\mathcal{D}}_\alpha = \left| \overrightarrow{\mathcal{C}}_1 \odot \overrightarrow{\mathcal{X}}_\alpha - w * \overrightarrow{\mathcal{X}} \right| \tag{15}$$

$$\overrightarrow{\mathcal{D}}_\beta = \left| \overrightarrow{\mathcal{C}}_2 \odot \overrightarrow{\mathcal{X}}_\beta - w * \overrightarrow{\mathcal{X}} \right| \tag{16}$$

$$\overrightarrow{\mathcal{D}}_\gamma = \left| \overrightarrow{\mathcal{C}}_3 \odot \overrightarrow{\mathcal{X}}_\gamma - w * \overrightarrow{\mathcal{X}} \right| \tag{17}$$

where w is the inertia constant weight. Inertia constant weight is used to control the exploration and exploitation of the algorithm. $\overrightarrow{\mathcal{A}}$ and $\overrightarrow{\mathcal{C}}$ i.e. coefficient vector and position vectors are calculated similarly as in the equation. Velocity and position are updated as follows: Velocity at time $(t+1)$ is given by

$$v^i(t+1) = v^i(t) * w + c_1 * r_1 * (X_1 - X) + c_2 * r_2 * (X_2 - X) + c_3 * r_3 * (X_3 - X). \tag{18}$$

The position is updated using the following:

$$X(t+1) = X^i(t) + v^i(t+1). \tag{19}$$

Feature selection is a multiobjective problem. One objective is not to minimize the number of features in the dataset and the second is to maximize the accuracy of the classifier. The fitness function is calculated as follows:

$$\text{Fitness} = 0.7 \times \text{accuracy(agent)} + 0.3 \times \frac{\text{tot\_feature} - \text{Sel\_feature}}{\text{tot\_feature}} \tag{20}$$

where tot_feature is total features, Sel_feature is selected features. Fig. 4 shows the flowchart for the proposed GWPSO feature selection approach. Algorithm 1 shows the procedure for training the proposed GWPSO feature selection approach.

Algorithm 2 GWPSO-Enhanced Graph Malware Detector (GWPSO-GAMD)

Input: Graph $G(V, E)$ with nodes V and edges E, Node features X, Labels Y
Output: Classification of nodes as benign or malicious
Initialize GWPSO parameters
Derive initial feature set F from X
▷ Feature Selection with GWPSO
Apply GWPSO on F to select optimal feature subset F_{opt}
Extract F_{opt} from X to form the optimized feature matrix X_{opt}
▷ Graph Neural Network Processing
Initialize GNN model with layers: GCN → GAT → GCN → GAT → Dense
Split data into training and testing sets: $X_{train}, X_{test}, Y_{train}, Y_{test}$
for each epoch **do**
Perform forward propagation through GNN layers
Calculate loss and update model parameters using Adam optimizer
▷ Model Evaluation
Evaluate model on X_{test} to predict $\hat{Y}_{test}$
Calculate performance metrics (e.g., accuracy, precision, recall)
return Model performance metrics

C. GWPSO-Enhanced Graph Android Malware Detector

The features obtained from GWPSO-based feature selection is used to design a GNN model tailored for the task of malware detection. The GWPSO method identifies the most relevant features within a complex malware dataset, which is represented as a graph where nodes correspond to system entities, and edges delineate interactions or relations among them. This model aims to classify nodes as benign or malicious by utilizing both the graph's topology and the node's feature information. Our architecture integrates GCN layers with GAT layers to effectively process the selected features and capture the intricate patterns indicative of malware. Algorithm 2 shows the proposed GWPSO-enhanced GAMD.

1) Model Architecture:

1) *Input Graph Representation:* The preprocessed graph data, incorporating features selected by the GWPSO technique, is denoted by its feature matrix $X \in \mathbb{R}^{N \times F}$, where N represents the number of nodes and F the number of selected features per node. The adjacency matrix $A \in \mathbb{R}^{N \times N}$ captures the connectivity among the nodes.
2) *GCN Layer:* The first layer in our model is a GCN layer, applying the operation

$$H^{(1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}XW^{(0)}\right) \tag{21}$$

where $\tilde{A} = A + I_N$ is the adjacency matrix with added self-loops, $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ is the degree matrix for $\tilde{A}$, $W^{(0)}$

denotes the weight matrix for this layer, and σ is the ReLU activation function.

3) *GAT Layer:* Following the GCN layer, we employ a GAT layer to dynamically assign importance to nodes within a neighborhood

$$H^{(2)} = \text{Concat}_{k=1}^{H}\left(\sigma\left(\sum_{j\in\mathcal{N}(i)}\alpha_{ij}^{(k)}W^{(k)}h_j^{(1)}\right)\right) \quad (22)$$

with H being the number of attention heads, $\mathcal{N}(i)$, including node i and its neighbors, $\alpha_{ij}^{(k)}$ as the learned attention coefficients for head k, $W^{(k)}$ the weight matrix for head k, and $h_j^{(1)}$ the feature vector of node j from $H^{(1)}$.

4) *Subsequent Layers and Output:* The model architecture further incorporates an additional GCN and GAT layer, enhancing the capability to discern both local and global structural patterns critical for malware classification. The processed features are subsequently passed through a Dropout layer for regularization and a Dense layer with a sigmoid activation function for binary classification.

5) *Model Compilation and Loss Function:* The model is compiled with the Adam optimizer, a popular choice for DL models due to its adaptive learning rate capabilities, mathematically represented as follows:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t}+\epsilon}\hat{m}_t \quad (23)$$

where θ represents the model parameters, η is the learning rate, $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected estimates of the first and second moments of the gradients, respectively, and ϵ is a small scalar added to improve numerical stability. For the loss function, binary cross-entropy is utilized, reflecting the model's focus on binary classification tasks. The loss for a single observation is given by

$$L(y,\hat{y}) = -\left[y\log(\hat{y}) + (1-y)\log(1-\hat{y})\right] \quad (24)$$

where y is the true label (0 for benign, 1 for malware) and $\hat{y}$ is the predicted probability of the node being malware.

2) Baseline Models: The proposed GWPSO-enhanced graph Android malware detector benchmarks its performance against established baselines that utilize well-known ensemble learning techniques: gradient boosting, bagging, and the extra tree classifier. These baseline methods involve training multiple learners to collaboratively address the challenge of malware detection. First, ensemble learning trains individual learners collectively using methods, such as maximum voting, averaging, or weighted averaging. This methodology is divided into two primary categories: 1) boosting, which operates in a sequential ensemble manner and 2) bagging, which functions as a parallel ensemble. Below are the baseline ensemble learning techniques employed for comparison in our model.

1) *Bagging*: This technique integrates adaptive boosting with weighted minimization, where base learners are sequentially created to progressively correct errors made by previous learners. The objective is to reduce the loss function via gradient descent, focusing on weak learners, the loss function itself, and an additive model. New predictors are incrementally added to minimize loss, ceasing when a predefined number of predictors is reached or when further iterations do not reduce loss.
2) *Gradient boosting*: This parallel ensemble method trains predictors independently and in parallel, facilitating simultaneous training across different machines and CPUs. The original dataset is divided into multiple subsets, each containing a random selection of samples, which may include duplicates. Base learners are then independently trained on these subsets in parallel, with their outcomes aggregated through a voting process.
3) *Extra Tree Classifier*: Primarily utilized for supervised learning and regression problems, this ensemble technique hinges on three parameters: the number of decision trees (DT), the number of features randomly selected, and the minimum number of instances required to split a node. The technique combines multiple models to enhance performance, with the best outcomes often achieved when the base learners operate independently.

IV. Results and Discussion

The proposed GWPSO-GAMD framework was developed on an Intel Xeon Gold 5220 CPU with two processors, with 256 GB of RAM, and was operated under Windows Server 2019 standard. We have used Python programming language (ver 3.9.18) and Jupyter Notebook to develop the GWPSO-GAMD framework. Additionally, Python libraries "NumPy" and "Pandas" were used for data manipulation and analysis. The GAMD was designed using the "Spektral" library within the TensorFlow (ver 2.10.1) ecosystem. This architecture used a GCN layer with 32 units and a ReLU activation function, and a GAT layer with 32 units, eight attention heads, and ReLU activation. The `concat_heads=True` parameter was used to perform concatenation of the output of the attention heads. A dropout layer with a rate of 0.5 was used to prevent overfitting by randomly setting a fraction of input units to 0 at each update during the training time, which helps to make the model more robust to unseen data. After processing through the GCN and GAT layers, the output was flattened to transform the graph data into a format suitable for standard neural network processing. Optimization and loss computation were handled by the "Adam" optimizer and "BinaryCrossentropy," respectively, with binary accuracy serving as the metric for model performance evaluation. To ensure a fair comparison, all baseline models, including 1-D-CNN, LSTM, GRU, BiGRU, and CNN-LSTM, and state-of-the-art techniques were implemented and evaluated under the same hardware and software configuration.

A. Description of Dataset

1) CIC-MalDroid-2020: [27] is a comprehensive collection of Android malware, designed to support the development and testing of malware detection systems. This data has 11598 instances and 470 features collected from various sources, such as VirusTotal and Contagio security blog among

others, covering a period from December 2017 to December 2018. The dataset consists of five categories of malware: 1) riskware; 2) SMS malware; 3) banking malware; 4) adware; and 5) benign. However, in this experiment, we have combined all types of malware in one group and performed a binary classification task.

2) CIC-MalMem-2022: [28] dataset serves as a complementary resource, focusing on the malware's memory footprint and behaviors to facilitate a deeper understanding of malicious software's operational dynamics. This dataset is instrumental in advancing research in malware detection by offering a rich set of features extracted from dynamic analysis, including API calls, memory usage, and network activity. This dataset has 58 596 samples and 56 features. Dataset consists of two categories: 1) benign with 29298 instances and 2) malware with 29298 instances.

B. Evaluation Metrics

Evaluation metrics used to evaluate the proposed GWPSO-GAMD framework are accuracy, precision, recall, and f1 score. These are calculated using True Positive ($\mathbb{P}\alpha$), False Negative ($\mathbb{N}\alpha$), False Positive ($\mathbb{P}\beta$), and True Negative ($\mathbb{N}\beta$). Evaluation metrics are calculated as follows.

1) Precision: It determines the number of correctly predicted malware samples divided by the total number of attacks observed by model

$$\text{Precision} = \frac{\mathbb{P}\alpha}{\mathbb{P}\alpha + \mathbb{P}\beta}. \tag{25}$$

2) Recall: It is also known as the detection rate. Determines the number of correctly predicted samples divided by the total number of active test cases

$$\text{Recall} = \frac{\mathbb{P}\alpha}{\mathbb{P}\alpha + \mathbb{N}\alpha}. \tag{26}$$

3) Accuracy: It is the ratio of correctly predicted samples by model to the total number of test cases observed. It is the probability that the prediction made by the model is correct

$$\text{Accuracy} = \frac{\mathbb{P}\alpha + \mathbb{N}\beta}{\mathbb{P}\alpha + \mathbb{N}\alpha + \mathbb{N}\beta + \mathbb{P}\beta}. \tag{27}$$

4) F1-Score: It is more useful than accuracy when the dataset is imbalanced. It is the mean of precision and recall. Its best value is 1 and worst is 0

$$\text{F1 Score} = 2\left(\frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}\right). \tag{28}$$

C. Result Analysis

This subsection gives a detailed explanation related to the performance of the proposed GWPSO-GAMD framework. First, we applied the preprocessing steps to the CIC-MalDroid-2020 [27] and CIC-MalMem-2022 [28] datasets. The initial phase of preprocessing involved handling missing values and encoding categorical variables, essential steps to prepare the datasets for subsequent analysis. Given the diverse nature of Android malware data, missing values were identified and treated to maintain the integrity of the datasets. This process included strategies such as imputation, which missing entries were replaced with statistically relevant values, ensuring that no data point was ignored due to incompleteness. Simultaneously, categorical variables within the datasets underwent encoding procedures. Given that the GWPSO-GAMD framework inherently requires numerical input, categorical features representing various characteristics of malware, such as application permissions or API calls, were transformed into a numerical format. This transformation was pivotal in retaining the categorical data's informative essence while making it interpretable for the proposed approach.

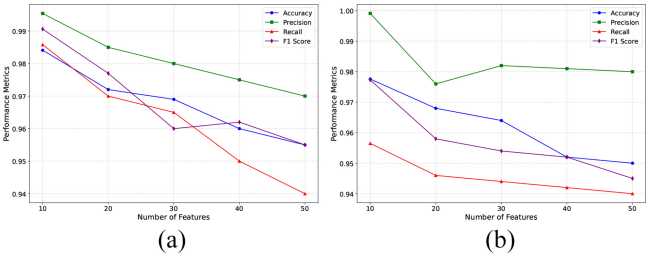

Fig. 5. Performance of GWPSO-GAMD framework across different feature counts (a) Performance using CIC-MalDroid-2020 dataset (b) Performance using CIC-MalMem-2022 dataset.

Next, as illustrated in Fig. 5, we evaluated the GWPSO-GAMD framework with subsets of features 50, 40, 30, 20, and 10 on the CIC-MalDroid-2020 and CIC-MalMem-2022 datasets, respectively. For instance, as illustrated in Fig. 5(a), it shows a progressive improvement in the GWPSO-GAMD framework's accuracy, precision, recall, and F1 score as the number of features diminishes to ten using the CIC-MalDroid-2020 dataset. Specifically, the framework's metrics with ten features stand as follows: Accuracy at 0.9776, Precision at 0.9991, Recall at 0.9565, and F1 Score at 0.9773. Similarly, Fig. 5(b) reveals the performance of the framework in the CIC-MalMem-2022 dataset for the same subsets of features. A reduction in feature count to ten achieves notable enhancements across all metrics: accuracy to 0.9841, precision to 0.9954, recall to 0.9858, and F1 score to 0.9906. This underscores the GWPSO technique's success in pinpointing the most pertinent features for malware detection, thereby enhancing framework performance while simplifying computational demands. Moreover, the outlined performance metrics for both datasets confirm the GWPSO-GAMD framework's robustness and the pivotal role of the GWPSO feature selection technique in optimizing feature sets for enhanced malware detection.

Next, we used a confusion matrix to assess the performance of the GWPSO-GAMD framework in malware detection tasks. This is shown in Fig. 6 for the CIC-MalDroid-2020 and CIC-MalMem-2022 datasets, respectively. A confusion matrix provides a comprehensive overview of a framework's predictive capabilities across actual and forecasted classifications, crucial for understanding its effectiveness in distinguishing between malware and benign instances. For the CIC-MalDroid-2020 dataset [Fig. 6(a)], the confusion matrix reveals that it correctly identified 1944 malware instances (True Positives) and 339 benign instances (True Negatives), while inaccurately classifying 9 benign instances as malware

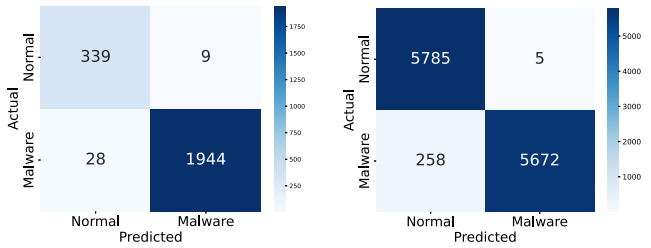

Fig. 6. Confusion matrices for GWPSO-GAMD framework (a) Confusion matrix using CIC-MalDroid-2020 dataset (b) Confusion matrix using CIC-MalMem-2022 dataset.

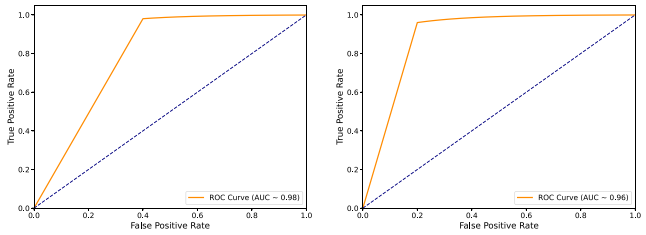

Fig. 7. AU-ROC Curve for GWPSO-GAMD framework (a) AU-ROC Curve using CIC-MalDroid-2020 dataset (b) AU-ROC Curve using CIC-MalMem-2022 dataset.

(False Positives) and failing to detect 28 malware instances (False Negatives). Similarly, Fig. 6(b) reveals the confusion matrix for the CIC-MalMem-2022 dataset. It reveals that the framework accurately identified 5672 malware instances and 5785 benign instances, showcasing its effectiveness in distinguishing between malicious and normal samples. Conversely, the framework inaccurately classified five benign instances as malware and failed to detect 258 malware instances. These results highlight the framework's precision in correctly identifying a vast majority of both malware and benign instances, with a very low rate of false alarms. The presence of false negatives, while relatively low, points to areas for further refinement to enhance the framework's sensitivity and ensure more comprehensive malware detection.

The receiver operating characteristic (ROC) curve is a fundamental tool used in evaluating the performance of binary classification models, depicting the tradeoff between the true positive rate (TPR) and false positive rate (FPR) across a range of thresholds. It's especially useful in contexts where the distinction between classes (such as malware versus benign instance) is critical. The AUC provides a single measure summarizing the model's ability to discriminate between positive and negative classes across all thresholds, with a value of 1 indicating perfect discrimination and 0.5 suggesting no discriminative power. Fig. 7 shows the AU-ROC Curve for the GWPSO-GAMD framework using CIC-MalDroid-2020 and CIC-MalMem-2022 datasets, respectively. For the CIC-MalDroid-2020 dataset [Fig. 7(a)], the AUC is 0.98 and for the CIC-MalMem-2022 [Fig. 7(b)] AUC is 0.96. Overall, the ROC curves and their respective AUC values for both datasets underscore the GWPSO-GAMD framework's strength in classifying and detecting malware.

TABLE III
PREDICTION RESULTS COMPARISONS WITH BASELINES USING CIC-MALDROID-2020

Feature Selection Method	Classification Algorithm	Accuracy	Precision	Recall	F1-Score
Without feature Selection	Bagging	93.20	91.60	91.90	91.89
	Gradient Boosting	92.90	92.20	91.40	91.72
	Extra Tree	94.25	95.14	94.55	94.78
	Proposed GAMD	95.10	96.56	94.10	94.78
Grey Wolf Optimizer (GWO)	Bagging	92.30	90.60	90.90	90.70
	Gradient Boosting	91.63	90.30	90.10	90.10
	Extra Tree	93.77	92.97	92.15	93.18
	Proposed GAMD	94.22	93.78	93.90	93.18
Particle Swarm Optimization (PSO)	Bagging	92.50	90.70	90.90	90.80
	Gradient Boosting	91.60	90.40	89.99	90.10
	Extra Tree	93.00	93.00	93.00	93.00
	Proposed GAMD	94.10	92.55	92.90	92.54
Proposed GWPSO	Bagging	93.20	92.50	91.16	91.50
	Gradient Boosting	92.00	92.90	90.30	90.60
	Extra Tree	96.10	95.40	93.55	94.10
	Proposed GAMD	98.41	99.54	98.58	99.06

TABLE IV
PREDICTION RESULTS COMPARISONS WITH BASELINES USING CIC-MALMEM-2022

Feature Selection Method	Classification Algorithm	Accuracy	Precision	Recall	F1-Score
Without feature Selection	Bagging	76.20	55.80	55.40	55.50
	Gradient Boosting	71.40	47.10	46.40	45.60
	Extra Tree	75.10	53.40	53.30	53.35
	Proposed GAMD	88.15	89.02	86.32	86.10
Grey Wolf Optimizer (GWO)	Bagging	73.30	50.51	50.21	50.15
	Gradient Boosting	68.60	42.64	41.01	39.97
	Extra Tree	73.30	50.24	50.00	49.99
	Proposed GAMD	87.25	88.20	85.91	84.01
Particle Swarm Optimization (PSO)	Bagging	74.70	53.10	52.80	51.80
	Gradient Boosting	69.00	42.50	42.10	41.90
	Extra Tree	75.00	54.60	54.44	54.30
	Proposed GAMD	86.15	87.31	85.25	87.12
Proposed GWPSO	Bagging	86.54	74.50	78.16	77.50
	Gradient Boosting	80.00	72.10	74.80	93.20
	Extra Tree	89.30	80.20	86.10	84.00
	Proposed GAMD	97.76	99.91	95.65	97.73

D. Performance Analysis of GWPSO-GAMD for Carbon-Intelligent IIoT Security

Performance evaluation of the GWPSO-GAMD framework, demonstrated in Fig. 8, highlights its computational efficiency and suitability for carbon-aware IIoT environments. The Feature Extraction Time graph shows that leveraging optimized feature selection (ten features) significantly reduces extraction time, making the framework more adaptable for energy-efficient edge computing. Similarly, the Training Time graph indicates that complete feature sets lead to higher computational costs, while optimized features reduce the training time by approximately 67.5%, which is crucial for low-power AI-driven security solutions in IIoT. The Inference time analysis further supports this claim, showing a 3× speed-up in malware detection with optimized features, facilitating real-time threat mitigation in energy-constrained IIoT ecosystems. Additionally, the CPU Usage and Memory Consumption graphs reveal that optimized feature selection results in substantial reductions in computational overhead, lowering CPU usage by nearly 47% and memory footprint by 57%, thus enhancing the feasibility of deploying carbon-efficient, AI-driven security mechanisms on lightweight IoT and edge devices.

1) Comparison With Baselines: Tables III and IV show the comparative analysis of the prediction results using the CIC-MalDroid-2020 and CIC-MalMem-2022 datasets, respectively. Specifically, for the CIC-MalDroid-2020 dataset, the GWPSO-GAMD framework presented remarkable results with an accuracy of 98.41%, precision of 99.54%, recall of 98.58%, and an f1-Score of 99.06%. This indicates a significant improvement over other feature selection methods and

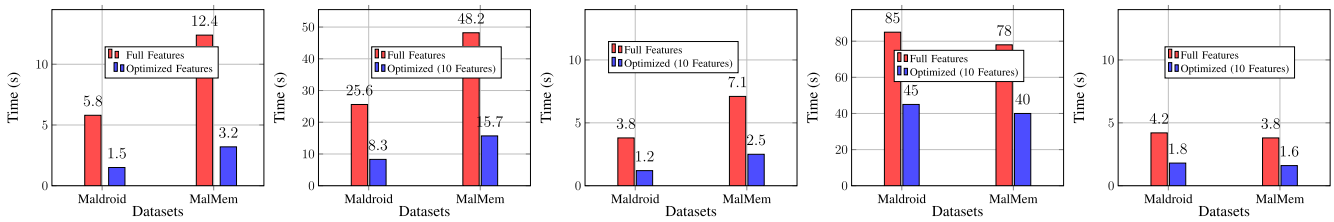

Fig. 8. Performance comparison of GWPSO-GAMD for energy-efficient malware detection, highlighting feature extraction, training, and inference times, along with CPU and memory usage metrics to demonstrate computational efficiency and suitability for resource-constrained IIoT environments.

TABLE V
PERFORMANCE COMPARISON (%) OF GWPSO-GAMD WITH DL MODELS ON CIC-MALDROID-2020 AND CIC-MALMEM-2022

Model	Dataset	Accuracy	Precision	Recall	F1-Score
1D-CNN	CIC-MalDroid-2020	91.35	89.72	88.91	89.31
LSTM	CIC-MalDroid-2020	93.84	92.41	91.67	92.04
GRU	CIC-MalDroid-2020	94.22	93.16	92.53	92.84
BiGRU	CIC-MalDroid-2020	95.76	94.88	94.01	94.44
CNN-LSTM	CIC-MalDroid-2020	96.28	95.47	94.89	95.18
GWPSO-GAMD	CIC-MalDroid-2020	98.41	99.54	98.58	99.06
1D-CNN	CIC-MalMem-2022	89.17	87.42	86.90	87.16
LSTM	CIC-MalMem-2022	91.56	90.13	89.67	89.90
GRU	CIC-MalMem-2022	92.84	91.21	90.45	90.83
BiGRU	CIC-MalMem-2022	94.18	93.30	92.62	92.96
CNN-LSTM	CIC-MalMem-2022	95.12	94.07	93.39	93.73
GWPSO-GAMD	CIC-MalMem-2022	97.76	99.91	95.65	97.73

TABLE VI
COMPARISON OF ACCURACY (%) WITH OTHER EXISTING TECHNIQUES USING CIC-MALDROID-2020

Reference	Detection Technique	Accuracy
Carrier *et al.* [28]	Ensemble Technique: NB, RF, DT as base learner and LR as meta learner	84.50
Issakhani *et al.* [28]	Ensemble Technique: RF, SVM, MLP, AdaBoost as base learner and LR as meta learner	86.30
Dhalaria *et al.* [20]	Chi square + KNN_RF	88.90
Proposed approach	GWPSO-GAMD	98.41

TABLE VII
COMPARISON OF ACCURACY(%) WITH OTHER EXISTING TECHNIQUES USING CIC-MALMEM-2022

Reference	Detection Technique	Accuracy
Carrier *et al.* [28]	Ensemble Technique: NB, RF, DT as base learner and LR as meta learner	76.20
Issakhani *et al.* [28]	Ensemble Technique: RF, SVM, MLP, AdaBoost as base learner and LR as meta learner	76.20
Dhalaria *et al.* [20]	Chi-square + KNN_RF	50.80
Proposed approach	GWPSO-GAMD	97.76

classification algorithms listed, where the closest competing method, using Extra Tree without feature selection, achieved lower performance metrics across the board. Similarly, for the CIC-MalMem-2022 dataset, the GWPSO GAMD again outperformed other approaches with an accuracy of 97.76%, precision reaching almost perfection at 99.91%, recall at 95.65%, and f1-Score at 97.73%. This performance is notably superior compared to other feature selection methods and classification algorithms used in the study. The closest competitor, Extra Tree with GWPSO, showed lower performance, emphasizing the effectiveness of the GWPSO-GAMD framework. These tables underline the importance of the hybrid feature selection technique offered by GWPSO in enhancing the predictive capability of the GAMD framework. The results underscore not only the effectiveness of GWPSO as a feature selection method but also the robustness and reliability of the GAMD model in detecting malware across varied and complex datasets. The exceptional performance of the proposed GWPSO-GAMD framework highlights its potential as a leading solution in the domain of Android malware detection, providing a significant contribution to the field of cybersecurity.

2) Comparison With Other Deep Learning Techniques: Table V presents a comparative evaluation of the proposed GWPSO-GAMD model against several DL baselines, including 1-D-CNN, LSTM, GRU, BiGRU, and CNN-LSTM, using the CIC-MalDroid-2020 and CIC-MalMem-2022 datasets. Across both datasets, GWPSO-GAMD consistently outperforms all baseline models in terms of accuracy, precision, recall, and F1-score. On CIC-MalDroid-2020, GWPSO-GAMD achieves an accuracy of 98.41%, with precision, recall, and F1-score values reaching 99.54%, 98.58%, and 99.06%, respectively. Similarly, on CIC-MalMem-2022, it attains 97.76% accuracy, 99.91% precision, 95.65% recall, and a 97.73% F1-score. These results demonstrate the model's strong generalization capabilities and robustness across diverse malware detection scenarios, particularly in comparison to conventional sequential or hybrid DL architectures.

3) Comparison With State-of-the-Art Techniques: Tables VI and VII show the comparison between our proposed GWPSO-GAMD framework and various existing techniques based on the CIC-MalDroid-2020 and CIC-MalMem-2022 datasets, respectively. The proposed GWPSO + GAMD method significantly outperforms other cited techniques with an impressive accuracy of 98.41%. This is a substantial improvement over ensemble techniques and combinations of ML models such as NB, RF, DT, SVM, multilayer perceptron (MLP), AdaBoost, and LR as meta-learner, which achieved accuracies ranging from 84.50% to 88.90%. These high performance values underline the effectiveness of our hybrid feature selection and classification framework in accurately detecting malware in the CIC-MalDroid-2020 dataset. Similarly, for the CIC-MalMem-2022 dataset, the proposed approach again demonstrates superior performance with an accuracy of 97.76%, substantially higher than competing methods, which range from 50.80% to 76.20%. This highlights the adaptability and robustness of the GWPSO-GAMD framework to deal with different types of datasets and challenges in malware detection.

The proposed GWPSO-GAMD framework was designed to address the unique challenges and threats that emerge within the interconnected and data-intensive environment of the industrial Metaverse, particularly focusing on mobile

ecosystems. In our rapidly evolving digital era, mobile devices and applications play a critical role in managing operations, data flow, and user interactions. However, this centrality makes them susceptible to malware attacks, which have the potential to jeopardize not only individual data privacy but also the integrity and reliability of large-scale industrial operations. The proposed framework tackles these challenges by integrating advanced feature selection based on the hybrid GWO and PSO named "GWPSO" and applying a novel graph-based analysis through the GAMD. This innovative approach addresses directly the complexities posed by mobile malware detection. By intelligently reducing the vast array of data to the most indicative features of malware, the framework significantly reduces computational burden, allowing for more efficient and rapid malware detection even in resource-constrained mobile environments. This optimized feature selection is crucial for maintaining the operational speed and responsiveness required in the fast-paced industrial network. Moreover, the utilization of graph-based analysis to examine the relational dynamics between features enables a deeper understanding of malware behavior. By capturing the complex interconnections and patterns within the data, the framework ensures a comprehensive security posture, safeguarding against sophisticated attacks that might exploit the interconnected nature of devices and applications within the mobile ecosystem.

V. Conclusion

This work presents GWPSO-GAMD, a resource-efficient, graph-based malware detection framework designed to enhance software security in IIoT networks. By integrating GWPSO for feature selection with a GAMD, the framework effectively reduces computational overhead while maintaining high detection accuracy. The experimental results on the CIC-MalDroid-2020 and CIC-MalMem-2022 datasets validate the effectiveness of GWPSO-GAMD, achieving high detection accuracies of 98.41% and 97.76%, respectively, while significantly reducing CPU usage, memory footprint, and processing time. The reduction in computational complexity and resource consumption demonstrates the feasibility of the real-time malware detection framework in energy-constrained IIoT environments. These improvements are particularly critical for scalable and decentralized IIoT applications, where traditional malware detection approaches often fail due to high dimensionality and computational costs. As future work, we aim to enhance the adaptability and security resilience of GWPSO-GAMD by integrating blockchain-based trust mechanisms for secure data sharing and federated learning for privacy-preserving malware detection. These enhancements will further strengthen IIoT cybersecurity, ensuring scalable, real-time, and autonomous threat detection for next-generation IIoT ecosystems.

References

[1] M. R. Babaei Mosleh and S. Sharifian, "An efficient cloud-integrated distributed deep neural network framework for IoT malware classification," *Future Gener. Comput. Syst.*, vol. 157, pp. 603–617, Aug. 2024.

[2] Y. Zhang, G. Gui, and S. Mao, "A lightweight malware traffic classification method based on a broad learning architecture," *IEEE Internet Things J.*, vol. 10, no. 23, pp. 21131–21132, Dec. 2023.

[3] S. H. Khan et al., "A new deep boosted CNN and ensemble learning based IoT malware detection," *Comput. Security*, vol. 133, Oct. 2023, Art. no. 103385.

[4] F. Nawshin, D. Unal, M. Hammoudeh, and P. N. Suganthan, "AI-powered malware detection with differential privacy for zero trust security in Internet of Things networks," *Ad Hoc Netw.*, vol. 161, Aug. 2024, Art. no. 103523.

[5] X. Zhang, L. Hao, G. Gui, Y. Wang, B. Adebisi, and H. Sari, "An automatic and efficient malware traffic classification method for secure Internet of Things," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8448–8458, Mar. 2024.

[6] Y. Qiao, W. Zhang, X. Du, and M. Guizani, "Malware classification based on multilayer perception and word2vec for IoT security," *ACM Trans. Internet Technol.*, vol. 22, no. 1, pp. 1–22, 2021.

[7] Z.-U. Rehman et al., "Machine learning-assisted signature and heuristic-based detection of malwares in android devices," *Comput. Elect. Eng.*, vol. 69, pp. 828–841, Jul. 2018.

[8] P. Musikawan, Y. Kongsorot, I. You, and C. So-In, "An enhanced deep learning neural network for the detection and identification of android malware," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8560–8577, May 2023.

[9] S. K. Smmarwar, G. P. Gupta, S. Kumar, and P. Kumar, "An optimized and efficient android malware detection framework for future sustainable computing," *Sustainable Energy Technologies and Assessments*, vol. 54, Dec. 2022, Art. no. 102852.

[10] H. Zhang, W. Zhang, Z. Lv, A. Kumar Sangaiah, T. Huang, and N. Chilamkurti, "MALDC: A depth detection method for malware based on behavior chains," *World Wide Web*, vol. 23, no. 2, pp. 991–1010, 2020.

[11] Y. Fang, Y. Yao, X. Lin, J. Wang, and H. Zhai, "A feature selection based on genetic algorithm for intrusion detection of industrial control systems," *Comput. Security*, vol. 139, Apr. 2024, Art. no. 103675.

[12] S. Yu, O. Jin, Y. Shen, G. Wu, S. Yu, and S. Shen, "Availability evaluation of Industrial Internet of Things under malware propagation: An extended reliability block diagram approach based on stochastic games," *IEEE Trans. Rel.*, early access, Aug. 8, 2024, doi: 10.1109/TR.2024.3434593.

[13] F. Jiménez, G. Sánchez, J. Palma, and G. Sciavicco, "Three-objective constrained evolutionary instance selection for classification: Wrapper and filter approaches," *Eng. Appl. Artif. Intell.*, vol. 107, Jan. 2022, Art. no. 104531.

[14] J. Chong, P. Tjurin, M. Niemelä, T. Jämsä, and V. Farrahi, "Machine-learning models for activity class prediction: A comparative study of feature selection and classification algorithms," *Gait Posture*, vol. 89, pp. 45–53, Sep. 2021.

[15] A. Thakkar and R. Lohiya, "A survey on intrusion detection system: feature selection, model, performance measures, application perspective, challenges, and future research directions," *Artif. Intell. Rev.*, vol. 55, pp. 453–563, Jan. 2022.

[16] M. Abdel-Basset, D. El-Shahat, I. El-henawy, V. H. C. de Albuquerque, and S. Mirjalili, "A new fusion of grey wolf optimizer algorithm with a two-phase mutation for feature selection," *Expert Syst. Appl.*, vol. 139, Jan. 2020, Art. no. 112824.

[17] S. Sundaramurthy and P. Jayavel, "A hybrid grey wolf optimization and particle swarm optimization with C4.5 approach for prediction of rheumatoid arthritis," *Appl. Soft Comput.*, vol. 94, Sep. 2020, Art. no. 106500.

[18] R. O. Ogundokun, J. B. Awotunde, P. Sadiku, E. Abidemi Adeniyi, M. Abiodun, and O. I. Dauda, "An enhanced intrusion detection system using particle swarm optimization feature extraction technique," *Procedia Comput. Sci.*, vol. 193, pp. 504–512, Nov. 2021.

[19] B. Sathiyabhama et al., "A novel feature selection framework based on grey wolf optimizer for mammogram image analysis," *Neural Comput. Appl.*, vol. 33, no. 21, pp. 14583–14602, 2021.

[20] M. Dhalaria and E. Gandotra, "Android malware detection using chi-square feature selection and ensemble learning method," in *Proc. 6th Int. Conf. Parallel, Distrib. Grid Comput. (PDGC)*, 2020, pp. 36–41.

[21] A. Fatima, R. Maurya, M. K. Dutta, R. Burget, and J. Masek, "Android malware detection using genetic algorithm-based feature selection," in *Data Intelligence and Cognitive Informatics*. Singapore: Springer, 2022, pp. 383–392.

[22] A. Mahindru and A. L. Sangal, "Hybridroid: An empirical analysis on effective malware detection model developed using ensemble methods," *J. Supercomput.*, vol. 77, no. 8, pp. 8209–8251, 2021.

[23] N. Al Sarah, F. Yasmin Rifat, M. S. Hossain, and H. S. Narman, "An efficient android malware prediction using ensemble machine learning algorithms," *Procedia Comput. Sci.*, vol. 191, pp. 184–191, Sep. 2021.
[24] H. Zhu, Y. Li, R. Li, J. Li, Z. You, and H. Song, "SEDMDroid: An enhanced stacking ensemble framework for android malware detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 984–994, Apr.–Jun. 2020.
[25] P. Sy Nguyen, T. N. Huy, T. Anh Tuan, P. Duy Trung, and H. Viet Long, "Hybrid feature extraction and integrated deep learning for cloud-based malware detection," *Comput. Security*, vol. 150, Mar. 2025, Art. no. 104233.
[26] Y. K. Sharma, D. S. Tomar, R. K. Pateriya, and S. Bhandari, "MOSDroid: Obfuscation-resilient android malware detection using multisets of encoded opcode sequences," *Comput. Security*, vol. 152, May 2025, Art. no. 104379.
[27] S. Mahdavifar, A. F. Abdul Kadir, R. Fatemi, D. Alhadidi, and A. A. Ghorbani, "Dynamic android malware category classification using semi-supervised deep learning," in *Proc. IEEE Int. Conf. Dependable*, 2020, pp. 515–522.
[28] T. Carrier, P. Victor, A. Tekeoglu, and A. H. Lashkari, "Detecting obfuscated malware using memory feature engineering," in *Proc. ICISSP*, 2022, pp. 177–188.

Govind P. Gupta (Member, IEEE) received the Ph.D. degree in computer science & engineering from the Indian Institute of Technology Roorkee, Roorkee, India, in 2014.

He is currently working as an Associate Professor with the Department of IT, National Institute of Technology Raipur, Raipur, India. He has more than 15 years of teaching and research experience. He has authored more than 120 research articles in peer-reviewed journals and conferences of international reputes. He has received several R&D projects from prestigious funding organizations, such as SERB (MATRIX), SERB(CRG), and C3iHub (IIT Kanpur). His main research interests include Internet of Things (IoT), IoT security, cyber threat intelligence, big data analytics and applied machine, and deep learning.

Dr. Gupta is a professional member of the ACM.

Prabhat Kumar (Member, IEEE) received the Ph.D. degree in information technology from the National Institute of Technology Raipur, Raipur, India, under the prestigious fellowship of the Ministry of Human Resource and Development, Government of India.

He is a Docent of Artificial Intelligence for Cybersecurity and a Postdoctoral Researcher with the Department of Software Engineering, LUT University, Lappeenranta, Finland, where he also serves as the Head of the Responsible Information Systems Development Master's Programme. He has authored or coauthored over 70+ top-tier journal/conference articles, including 15+ IEEE Transactions papers. He has received over 10 million Euros funding from CHIST-ERA/Research Council of Finland and HORIZON. He has many research contributions in computational design science, machine learning, deep learning, federated learning, big data analytics, cybersecurity, blockchain, cloud computing, Internet of Things, digital twin, and SDN.

Dr. Kumar was honored with the Best Postdoctoral Researcher Award (2023) by the Research Foundation of LUT University, Finland, through the Lauri ja Lahja Hotinen Fund, in addition.

Ahamed Aljuhani received the Ph.D. degree in computer science/information security track from The Catholic University of America, Wahington, DC, USA.

He is currently an Associate Professor and the Chair of the Department of Computer Engineering, College of Computing and Information Technology, University of Tabuk, Tabuk, Saudi Arabia. His research interests include information security, network security and privacy, secure system design, and system development.